

HOJA DE CONTENIDO DEL INFORME

Materia prima para tu informe futurista — constrúyelo en inglés

Hoja de contenido: materia prima del informe futurista

Estudiante	Cristian Navas
Proyecto	Signal Desk
Pitch	Un sistema de monitoreo en tiempo real para priorizar y gestionar eficientemente incidentes y alertas operativas.
Entorno	MAUI SDK .NET 10
Medio del informe	Word exportado a PDF
Fecha	2026-09-04
Docente	Ing. Javier Arturo García Mineros

Recordatorio: esta hoja no es el informe. Transforme este contenido en su PDF futurista siguiendo el brief de diseño.

2. El entorno utilizado

Eficiencia Multiplataforma (.NET MAUI): Permite compilar aplicaciones nativas para iOS, Android y escritorio desde un único código base utilizando C# y XAML. Esto reduce drásticamente el tiempo de desarrollo, facilita el mantenimiento del código y aprovecha el ecosistema maduro de .NET sin sacrificar el rendimiento nativo de los dispositivos. Agilidad en el Backend (Firebase): Al utilizar un modelo de Backend-as-a-Service (BaaS), Firebase elimina la necesidad de configurar, mantener y escalar servidores físicos o virtuales. Servicios integrados como Firestore (base de datos en tiempo real) y Firebase Authentication permiten implementar funciones complejas de forma rápida, reduciendo significativamente el time-to-market del proyecto.

2.1 Alternativa no utilizada

Flutter (Dart) Por qué: Aunque Flutter es un framework multiplataforma muy capaz, requiere aprender y adoptar el lenguaje Dart. En su lugar, se seleccionó .NET MAUI para aprovechar la sólida experiencia que ya tiene el equipo con C# y para lograr una integración perfecta con las bibliotecas estándar de .NET. Esto evita una curva de aprendizaje pronunciada y acelera el desarrollo inicial.

3. Herramientas

HERRAMIENTA	VERSIÓN	PARA QUÉ SE UTILIZÓ
.NET MAUI	10.0.100	Nuestro framework multiplataforma para la creación y el desarrollo del sistema utilizando C# y XAML.
Blazor Bootstrap	4.0.0 Versión estable	Para implementar interfaces prediseñadas sin necesidad de escribir código manualmente.
Firebase Console (SQL Connect)	17.3.2	Administrador de base de datos.

4. Recursos

TIPO DE RECURSO	DETALLE	CONTRIBUCIÓN AL PROYECTO
Hardware	Laptop de desarrollo con Windows y smartphone (Android/iOS) para pruebas.	Equipo principal utilizado para escribir el código, compilar la aplicación para diferentes plataformas y evaluar el rendimiento nativo.
Tiempo invertido	Total estimado de 3 horas y 17 minutos.	Esfuerzo de desarrollo necesario para completar el ciclo de vida de la aplicación, desde el diseño hasta la corrección de errores.
Documentación / tutoriales	Documentación oficial de Microsoft Learn para .NET MAUI.	Proporciona la arquitectura de referencia, buenas prácticas y ejemplos de código para implementar la interfaz de usuario.
Bibliotecas o paquetes	Paquetes NuGet específicos como Plugin.Firebase, FirebaseAuthentication.net y CommunityToolkit.Maui.	Reduce el tiempo de desarrollo al incorporar funciones de autenticación, sincronización de datos y componentes visuales sin programarlos desde cero.
IA o asistencia utilizada	Modelos de lenguaje grandes (LLM) como Gemini, ChatGPT y Claude.	Agiliza la resolución de errores, ayuda a diseñar las vistas XAML y contribuye a estructurar consultas complejas para la base de datos de Firebase.

5. Funcionalidad

FUNCIONALIDAD	QUÉ HACE Y VE EL USUARIO
1- Registro de tickets	El usuario completa el título, la descripción y selecciona un nivel de prioridad (P1-P4). Al hacer clic en "Registrar ticket", aparece una nueva entrada en la lista principal con fechas de vencimiento de SLA calculadas automáticamente.
2- Flujo de estados	El usuario selecciona un ticket en el panel y hace clic en "Marcar como resuelto" o actualiza su estado. La etiqueta de estado se actualiza dinámicamente de Nuevo/En progreso a Resuelto.
3- Filtrado por prioridad	El usuario selecciona una opción de filtro de prioridad en el panel. La lista se actualiza al instante para mostrar solo los tickets que coinciden con el nivel de prioridad seleccionado.
4- Persistencia de datos	El usuario cierra y vuelve a abrir la aplicación. Todos los tickets registrados previamente y los registros de resolución permanecen intactos y se cargan automáticamente desde el almacenamiento local en JSON.

5.1 Fuera del alcance

La aplicación no admite la gestión de roles para múltiples usuarios ni la sincronización con un backend en la nube. Tampoco envía notificaciones por correo electrónico o SMS cuando se vencen los SLA, ni se integra con herramientas de gestión de tickets de terceros como Jira o ServiceNow.

6. Arquitectura y modelo de datos

La aplicación utiliza el patrón arquitectónico MVVM (Modelo-Vista-ViewModel) en .NET MAUI. La carpeta Models contiene las estructuras de datos (Ticket.cs). La carpeta ViewModels se encarga de la lógica de presentación y los comandos (MainViewModel.cs). La carpeta Views contiene las páginas en XAML (MainPage.xaml), vinculadas a los ViewModels mediante Data Binding.

6.1 Persistencia de datos

Los datos se almacenan en memoria mediante una ObservableCollection<Ticket> mientras la aplicación está en ejecución, lo que permite actualizaciones rápidas de la interfaz de usuario. Para la persistencia, los datos de los tickets se serializan y se almacenan localmente en el disco en formato JSON (tickets.json).

Inserte aquí el diagrama de su modelo de datos.

7. Aspectos destacados del código

ARCHIVO Y MÉTODO	QUÉ HACE	POR QUÉ ES INTERESANTE
Ticket.cs – Modelo	Maneja la lógica de presentación y las acciones de la aplicación.	Aplica el patrón MVVM, separando la lógica de la aplicación de la interfaz de usuario.
MainViewModel.cs – Comandos	Mantiene los tickets en memoria y actualiza automáticamente la interfaz.	Permite que los cambios en los datos se reflejen rápidamente en la interfaz de usuario.
ObservableCollection<Ticket>	Conecta los elementos visuales con los datos y los comandos del ViewModel.	Reduce la necesidad de escribir código manual para controlar la interfaz.

Pegue aquí los tres fragmentos de código, con sus comentarios en inglés.

8. Registro de pruebas

ID	QUÉ SE PROBÓ	ENTRADA	RESULTADO ESPERADO	RESULTADO REAL	ESTADO
T-01	Creación de tickets	Verificar que la información del ticket se registre correctamente.	El ticket se crea y aparece en la lista.	El ticket se muestra correctamente en la lista.	Aprobado
T-02	Visualización de tickets	Verificar que los tickets se carguen correctamente en la interfaz.	Los tickets registrados aparecen correctamente.	Los tickets se muestran correctamente en la interfaz.	Aprobado
T-03	Actualización de tickets	Verificar que los cambios en un ticket se reflejen en la aplicación.	La información modificada se actualiza correctamente.	Los cambios se reflejan en la lista.	Aprobado
T-04	Eliminación de tickets	Verificar la eliminación de un ticket seleccionado.	El ticket seleccionado se elimina de la lista.	El ticket se elimina correctamente.	Aprobado
T-05	Actualizaciones de la interfaz	Verificar el funcionamiento de ObservableCollection<Ticket>.	La interfaz se actualiza automáticamente cuando cambian los datos.	La lista se actualiza correctamente.	Aprobado
T-06	Guardado en tickets.json	Verificar la persistencia de datos.	Los tickets se almacenan correctamente en el archivo JSON.	Los datos se guardan correctamente en formato JSON.	Aprobado
T-07	Carga de datos almacenados	Verificar que los tickets guardados puedan recuperarse.	Los tickets almacenados vuelven a aparecer al iniciar la aplicación.	Los datos guardados se recuperan correctamente.	Aprobado
T-08	Interfaz XAML y navegación	Verificar que las páginas y los controles funcionen correctamente.	La interfaz permite usar las funciones principales sin errores.	La navegación y los controles funcionan correctamente.	Aprobado

9. Retos y conclusiones

PROBLEMA	QUÉ BUSQUÉ EN INGLÉS	CÓMO LO RESOLVÍ
Necesitaba simular una base de datos antes de conectar el backend real.	"net maui mock data in memory observablecollection"	Creé una clase de servicio (MockDataService) con listas ObservableCollection para almacenar y gestionar datos temporalmente en memoria mientras la aplicación se ejecuta.
Los datos en memoria se reiniciaban al navegar entre ciertas páginas.	"net maui keep state singleton service dependency injection"	Registré mi servicio de datos simulado como Singleton en MauiProgram.cs para mantener un estado único y persistente durante toda la sesión de la aplicación.
Simular el tiempo de respuesta de una base de datos real para probar los indicadores de carga en la interfaz.	"c# simulate async network call task delay"	Agregué await Task.Delay(1000) a los métodos asíncronos de mi servicio en memoria para simular la latencia de red y verificar el comportamiento visual de la aplicación.

9.1 Conclusiones

En esta fase inicial del proyecto, nos enfocamos en construir un frontend multiplataforma robusto usando .NET MAUI y la arquitectura MVVM. Debido a que el backend de Firebase aún no está implementado, utilizamos un servicio de datos en memoria para simular las interacciones con la base de datos. Este enfoque nos permitió crear un prototipo rápido de la interfaz, probar el enlace de datos y perfeccionar la navegación sin depender de la configuración del servidor. El principal aprendizaje fue la importancia de programar utilizando interfaces (ej. `IDataStore`); de esta manera, reemplazar nuestro servicio simulado actual con el servicio real de Firebase en el futuro requerirá cambios mínimos en nuestros ViewModels.

Declaración

Declaro que escribí el código y el informe, que todo identificador y comentario está en inglés y que puedo explicar cualquier parte de este trabajo.

Cristian Navas
Student / Estudiante

Ing. Javier A. García Mineros
Teacher / Docente